

Types de données simples

Nombres (int, float)

Exemples: 0 42 -1 3.2 -0.3

Opérations de base: + - * /

Puissance: **

Parenthèses contrôlent l'ordre des opérations:

```
(1+2)*3
```

Division entière et reste: // %

Arrondi et troncature: round() int()

Fonctions maths: max() min() abs()

Booléens (bool)

Deux valeurs possibles: True False

Chaînes de caractères (str)

Chaîne: "bonjour"

Chaîne vide: ""

Concaténation de chaînes:

```
"bonjour " + "tout_le_monde"
```

Longueur: len("bonjour") (donne 7)

Conversions entre types

```
int() float() str() bool()
```

Exemples: x=int("42") s=str(42)

Variables

Un nom de variable commence par une lettre et ne contient pas d'espace.

Assignation simple: ma_variable=234.5

Assignation avec expression à droite:

```
a=b+c x=x+1
```

Afficher et Demander

Afficher: print(valeur1, valeur2, ...)

Demander: réponse=input("question")

Demander et convertir en une fois:

```
nb_entier=int(input("un nombre stp"))
```

Commentaires

Ils sont à destination des humains, python n'interprète pas les commentaires.

```
"""
```

```
Un bloc de commentaire,  
peut s'étendre sur plusieurs lignes.
```

```
"""
```

```
print(42) # Un commentaire en ligne
```

Structures de contrôle

Conditions

Simple: == != < <= > >= in

Combinées: and or not a < b < c

Branchements conditionnels if / else

```
if condition:  
    instructions si condition vraie  
else: # optionnel  
    instructions si condition fausse  
instructions inconditionnelles
```

Boucles while

```
while condition:  
    instructions qui se répètent  
instructions en dehors de la boucle
```

Listes

Liste: L=[-1, 3.2, 1/3, "bob"]

Liste vide: []

Crochets avec le clavier: Alt+5 Alt+6

Récupérer et enlever un élément de la fin:

```
dernier_element=L.pop()
```

Récupérer et enlever un élément du début:

```
premier_element=L.pop(0)
```

Ajouter x à la fin: L.append(x)

Insérer x à la position p: L.insert(p, x)

Insérer x au début: L.insert(0, x)

Savoir si une liste contient des éléments:

```
if L:  
    instructions
```

Itérer les éléments d'une liste du début à la fin:

```
while L:  
    element=L.pop(0)  
    instructions qui utilisent element
```

Savoir si un élément est dans une liste:

```
if element in L:  
    instructions
```

Affectation de liste: L2=L # même objet!

Copie de liste: L2=L.copy()

Fonctions listes: len() max() min() sum()